

Writeup: – OPERATION_ULTRA

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup_14_OperationUltraOK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `$JzIC18EX@BRtJk#`

Résumé

Et surtout : des noms intéressants comme `flag_part1`, `flag_part2`, `rc4_key`, `encrypted_values`, etc.

Le binaire il n'est pas stripé, donc on pourra explorer les symboles internes.

analyse des symboles internes

liste des symboles :

```
nm -C OPERATION_ULTRA
```

Outils Utilisés

Analyse effectuée avec les outils standards pour reverse engineering (objdump, gdb, strings, scripts personnalisés).

Analyse Statique

Extraction des chaînes de caractères

Première analyse rapide avec :

```
strings OPERATION_ULTRA
```

Il est probablement vérifié par un algorithme de chiffrement RC4

Inspection des sections ELF

Inspection plus approfondie

```
readelf -S OPERATION_ULTRA
```

On voit :

.text pour le code,

.data pour les variables importantes,

.bss pour les buffers dynamiques.

dump des données utiles

Analyse Dynamique

Enter the decryption key:

7Bi13pw

q;Te!

```
$JzIC18EX@BRtJk#Debugger detected! Nice try ;)
OPERATION_ULTRA.s
good_msg
good_len
ETC...
```

Des textes de contexte ("Bletchley Park", "Alan Turing", etc.),

Les messages Good Job!, Bad Password!,

Identification du Mécanisme de Validation

On vérifie rapidement son type :

```
file OPERATION_ULTRA
ELF 64-bit LSB executable, non stripé.
```

Découverte du Flag

PSH1

H=@B

Good Job! You've decrypted Enigma! Enemy messages are now readable, the war will soon be over!

Bad Password! Your decryption failed... The U-Boot submarines continue to sink Allied ships.

=== OPERATION ULTRA: BLETCHLEY PARK, 1942 ===

Station X, headquarters of the Government Code & Cypher School.

Alan Turing has recruited you to join the codebreakers team.

On veut lire la mémoire où se trouvent les fragments du flag :

lecture de flag_part1 à flag_part4 :

```
xxd -s 0x2290 -l 16 OPERATION_ULTRA
```

```
00002290: 244a 7a6c 4331 3845 5840 4252 744a 6b23
```

Qui correspond à l'ASCII :

```
$JzIC18EX@BRtJk#
```

Conclusion

Ce crackme a été résolu en comprenant son mécanisme de validation et en élaborant une stratégie appropriée.

Puis on regarde sa taille

Puis on regarde sa taille :

On observe

On observe :

PSH1

PSH1
VWQH1
PSQRVWH
PSH1
PSH1
VWQH1
PSQRVWH
_^ZY[X

PSQRVWAPAQI

PSQRVWAPAQI
AYAX_^ZY[X

Enemy communications use a sophisticated machine

Enemy communications use a sophisticated machine: Enigma.
This morning, a critical message was intercepted, probably orders
for the next submarine attack in the Atlantic.
Thousands of lives depend on your ability to crack this code...

On trouve

On trouve :

000000000000010 a flag_length
0000000000402290 d flag_part1
0000000000402294 d flag_part2
0000000000402298 d flag_part3
000000000040229c d flag_part4

...

Des variables en .data comme flag_part1, flag_part2, rc4_key, encrypted_values.
Des fonctions comme verify_fragment, init_rc4, rc4_crypt.

Ca suggère que

Ca suggère que :
Le flag est découpé en plusieurs morceaux,

Donc

Donc :
flag_part1 = "\$Jzl"

flag_part2 = "C18E"

flag_part3 = "X@BR"

flag_part4 = "tJk#"

le flag assemblé : \$JzIC18EX@BRtJk#